

Explicit Loading in Entity Framework Core

In this article, I will discuss **Explicit Loading in Entity Framework (EF) Core** with Examples. Please read our previous article discussing [Lazy Loading in Entity Framework Core](#) with Examples. At the end of this article, you will understand what explicit loading is and how to implement explicit loading in EF Core. We will work with the same example we have worked on so far.

What is Explicit Loading in Entity Framework Core?

Explicit Loading in Entity Framework Core is a technique where related entities are loaded manually after the main entity has already been retrieved from the database. Unlike Eager Loading (which loads related entities with the main entity in a single query) and Lazy Loading (which loads related entities automatically when the navigation property is accessed), Explicit Loading gives us full control over when and what related data to load, using methods like `Load()`. It requires explicit calls in the code to load related data, making it a manual process.

This allows us to defer loading related entities until they are needed. It is distinctly different from Lazy Loading because it requires explicit instructions to load the data rather than happening automatically. Explicit Loading is useful when related entities are conditionally required or when we want to minimize unnecessary data retrieval.

Example to Understand Explicit Loading in Entity Framework Core:

Let us understand this with an example. First, let us disable the Lazy loading for all the entities by setting the `LazyLoadingEnabled` property to false within the constructor of the context class or by removing the `UseLazyLoadingProxies()` method call from the `OnConfiguring` method of the `DbContext` class. So, first, modify the `EFCoreDbContext` class as shown below.

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        //Constructor calling the Base DbContext Class Constructor
        public EFCoreDbContext() : base()
        {
            //Disabling Lazy Loading
            this.ChangeTracker.LazyLoadingEnabled = false;
        }
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Enable Logging
            optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            //Connection String
            optionsBuilder.UseSqlServer("Server=LAPTOP-6P5NK25R\\SQLSERVER2022DEV;Database=StudentDB;Trusted_Connection=True;TrustSer");
            // DbSet properties represent the tables in the database.
            // Each DbSet corresponds to a table, and the type parameter corresponds to that table.
            public DbSet<Student> Students { get; set; }
            public DbSet<Teacher> Teachers { get; set; }
            public DbSet<Branch> Branches { get; set; }
        }
    }
}
```

```

        public DbSet<Address> Addresses { get; set; }
        public DbSet<Subject> Subjects { get; set; }
        public DbSet<Course> Courses { get; set; }
    }
}

```

Modifying the Program Class:

Next, modify the Program class as shown below to make sure Lazy Loading is Disabled and the related data is not loaded when we explicitly access the navigation properties.

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                try
                {
                    // Lazy Loading Example
                    Console.WriteLine("Lazy Loading Student and related
data\n");

                    // Load a student (only student data is loaded initially)
                    var student = context.Students.FirstOrDefault(s =>
s.StudentId == 1);

                    // Display basic student information
                    if (student != null)
                    {
                        Console.WriteLine($"Student Id:
{student.StudentId}, Name: {student.FirstName} {student.LastName}, Gender:
{student.Gender} \n");

                        // Check if Branch is null before accessing its
properties

                        if (student.Branch != null)
                        {
                            Console.WriteLine($"Branch Location:
{student.Branch.BranchLocation}, Email: {student.Branch.BranchEmail}, Phone:
{student.Branch.BranchPhoneNumber} \n");
                        }
                        else
                        {
                            Console.WriteLine("\nBranch data not
available.\n");
                        }

                        // Check if Address is null before accessing its
properties

                        if (student.Address != null)

```

```

        {
            Console.WriteLine($"Address:
{student.Address.Street}, {student.Address.City}, {student.Address.State},
Pin: {student.Address.PostalCode} \n");
        }
        else
        {
            Console.WriteLine("\nAddress data not
available.\n");
        }
    }
    else
    {
        Console.WriteLine("Student data not found.");
    }
}
catch (Exception ex)
{
    // Handle any errors that occur during data retrieval
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}
}
}

```

Now, run the application, and you should get the following output. As you can see, the related Branch and Address data are not available. This is because Lazy Loading is disabled, and we are not using Eager loading to load the related entities.

```

Lazy Loading Student and related data

info: 08-09-2024 14:12:30.648 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (25ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[FirstName], [s].[Gender], [s].[LastName]
      FROM [Students] AS [s]
      WHERE [s].[StudentId] = 1

Student Id: 1, Name: John Doe, Gender: Male

Branch data not available.

Address data not available.

```

How Do We Implement Explicit Loading in Entity Framework Core?

In Entity Framework Core (EF Core), Explicit Loading is a technique where related data is manually loaded after the primary entity has already been retrieved from the database. This is especially useful when we want more control over which related entities are loaded and when to load them. Explicit loading involves methods like `Entry()`, `Reference()`, `Collection()`, and `Load()`. Let us first understand each of these method and its role in Explicit Loading:

Entry() Method:

The `Entry()` method provides access to an entity's underlying details, such as its state and navigation properties. It returns an `EntityEntry` object for the specified entity, allowing us to inspect and modify the entity's state or explicitly load

navigation properties. This method is the entry point for performing explicit loading of navigation properties.

Example: `context.Entry(student);` // Get the entry for the student entity

Reference() Method:

The Reference() method is used to access a reference navigation property representing a single related entity (e.g., Branch or Address). This method is typically used when the related entity is not a collection. Once accessed, the Load() method is used actually to load the data.

Example: `context.Entry(student).Reference(s => s.Branch).Load();` // Load a single related entity (Branch)

Collection() Method:

The Collection() method accesses collection navigation properties, which represent collections of related entities (e.g., Courses related to a Student). This is useful when explicitly loading a collection of related entities. As with Reference(), we need to call the Load() method to fetch the actual data from the database.

Example: `context.Entry(student).Collection(s => s.Courses).Load();` // Load a collection of related entities (Courses)

Load() Method:

The Load() method retrieves related data from the database. It is used along with Reference() or Collection() to execute the actual query. The related entities will not be loaded from the database without invoking Load() method.

Example: `context.Entry(student).Reference(s => s.Branch).Load();` // Loads the related Branch entity

Example of Explicit Loading for a Single Entity:

Let's understand how to load a reference navigation property explicitly. Please modify the Program class as follows. Here, we are loading the Address and Branch explicitly.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                try
                {
                    Console.WriteLine("\nExplicit Loading Student Related
Data\n");

                    // Load a student (only student data is loaded initially)
                    var student = context.Students.FirstOrDefault(s =>
s.StudentId == 1);

                    // Display basic student information
                    if (student != null)
                    {
                        Console.WriteLine($"Student Id:
{student.StudentId}, Name: {student.FirstName} {student.LastName}, Gender:
{student.Gender} \n");
                    }
                }
            }
        }
    }
}
```

```

the student
s.Branch).Load();
properties
    // Explicitly load the Branch navigation property for
    context.Entry(student).Reference(s =>
    // Check if Branch is null before accessing its
    if (student.Branch != null)
    {
        Console.WriteLine($"Branch Location:
{student.Branch.BranchLocation}, Email: {student.Branch.BranchEmail}, Phone:
{student.Branch.BranchPhoneNumber} \n");
    }
    else
    {
        Console.WriteLine("\nBranch data not
available.\n");
    }
    // Explicitly load the Address navigation property
    context.Entry(student).Reference(s =>
    // Check if Address is null before accessing its
    properties
    if (student.Address != null)
    {
        Console.WriteLine($"Address:
{student.Address.Street}, {student.Address.City}, {student.Address.State},
Pin: {student.Address.PostalCode} \n");
    }
    else
    {
        Console.WriteLine("\nAddress data not
available.\n");
    }
    }
    else
    {
        Console.WriteLine("Student data not found.");
    }
}
catch (Exception ex)
{
    // Handle any errors that occur during data retrieval
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}
}
}

```

Explanation:

- The student data is loaded first without its related data (Branch and Address).
- The Branch and Address navigation properties are explicitly loaded using the Reference() method, and their properties are only accessed after checking for null values.

Now, run the application, and you should see the following output:

```
Explicit Loading Student Related Data

info: 08-09-2024 14:21:03.718 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (46ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [s].[StudentId], [s].[BranchId], [s].[DateOfBirth], [s].[FirstName], [s].[Gender], [s].[LastName]
      FROM [Students] AS [s]
      WHERE [s].[StudentId] = 1

Student Id: 1, Name: John Doe, Gender: Male

info: 08-09-2024 14:21:03.913 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (66ms) [Parameters=[@__p_0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [b].[BranchId], [b].[BranchEmail], [b].[BranchLocation], [b].[BranchPhoneNumber]
      FROM [Branches] AS [b]
      WHERE [b].[BranchId] = @__p_0

Branch Location: BBSR, Email: BBSR@dotnettutorials.net, Phone: 212-555-1234

info: 08-09-2024 14:21:03.960 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (10ms) [Parameters=[@__p_0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [a].[AddressId], [a].[City], [a].[PostalCode], [a].[State], [a].[Street], [a].[StudentId], [a].[TeacherId]
      FROM [Addresses] AS [a]
      WHERE [a].[StudentId] = @__p_0

Address: 123 Main St, New York, NY, Pin: 10001
```

Example of Explicit Loading for a Collection:

Let's understand how to explicitly load a collection navigation property. Please modify the Program class as follows. Here, we are explicitly loading a student's Courses collection.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                try
                {
                    Console.WriteLine("\nExplicit Loading Student Related
Data\n");

                    // Load a student (only student data is loaded initially)
                    var student = context.Students.FirstOrDefault(s =>
s.StudentId == 1);

                    // Display basic student information
                    if (student != null)
                    {
                        Console.WriteLine($"Student Id:
{student.StudentId}, Name: {student.FirstName} {student.LastName}, Gender:
{student.Gender} \n");
                    }
                }
            }
        }
    }
}
```

```

// Explicitly load the Courses collection for the
student
context.Entry(student).Collection(s =>
s.Courses).Load();
// Loop through the loaded courses and display course
names
foreach (var course in student.Courses)
{
    Console.WriteLine($"Course: {course.Name}");
}
else
{
    Console.WriteLine("Student data not found.");
}
}
catch (Exception ex)
{
    // Handle any errors that occur during data retrieval
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}
}

```

Explanation:

- The student data is first loaded, but the Course collection is not initially retrieved.
- Using the Collection() method, we explicitly load the Course collection for the student.
- The program then iterates through the collection and displays each course.

How EF Core Handles Explicit Loading?

When we explicitly load a related entity using methods like **Reference().Load()** or **Collection().Load()**, EF Core will check if the navigation property is already loaded in the current context.

- EF Core will not send another query if the navigation property is already loaded (i.e., its data is tracked in the DbContext).
- EF Core will send a SELECT query to retrieve the data if the navigation property is not loaded.

Let us understand this with an example. Please modify the Program class as follows:

```

using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())

```

```

    {
        try
        {
            Console.WriteLine("\nExplicit Loading Student Related
Data\n");
            // Load a student (only student data is loaded initially)
            var student = context.Students.FirstOrDefault(s =>
s.StudentId == 1);
            // Display basic student information
            if (student != null)
            {
                Console.WriteLine($"Student Id:
{student.StudentId}, Name: {student.FirstName} {student.LastName}, Gender:
{student.Gender} \n");
                // Explicitly load the Branch navigation property for
the student
                context.Entry(student).Reference(s =>
s.Branch).Load();
                // Check if Branch is null before accessing its
properties
                if (student.Branch != null)
                {
                    Console.WriteLine($"Branch Location:
{student.Branch.BranchLocation}, Email: {student.Branch.BranchEmail}, Phone:
{student.Branch.BranchPhoneNumber} \n");
                }
                else
                {
                    Console.WriteLine("\nBranch data not
available.\n");
                }
                // Explicitly load the Branch navigation property for
the student
                context.Entry(student).Reference(s =>
s.Branch).Load();
                // Check if Branch is null before accessing its
properties
                if (student.Branch != null)
                {
                    Console.WriteLine($"Branch Location:
{student.Branch.BranchLocation}, Email: {student.Branch.BranchEmail}, Phone:
{student.Branch.BranchPhoneNumber} \n");
                }
                else
                {
                    Console.WriteLine("\nBranch data not
available.\n");
                }
            }
        }
        else
        {

```


Note: Explicit Loading is not “another way to implement Lazy Loading,” as it requires the developer to explicitly decide and call for the related data to be loaded, providing more control over when and how related data is fetched.

In the next article, I will discuss [Default Conventions in Entity Framework Core](#) with Examples. In this article, I will try to explain **Explicit Loading in Entity Framework Core** with examples. I hope you enjoyed this article. Please provide your valuable feedback and suggestions for this article.